yearly_summary_query

```
SELECT
   year,
   SUM(total_usd) AS revenue,
   COUNT(order_id) AS orders_count,
   SUM(total_usd) / COUNT(order_id) AS aov,
   COUNT(DISTINCT customer_id) AS unique_customers
FROM orders
GROUP BY year
ORDER BY year;
```

yearly_revenue

```
SELECT
   year,month,
   SUM(total_usd) AS revenue
FROM orders
GROUP BY year,month
ORDER BY year,month;
```

revenue_monthly_change

```
SELECT
   month,
   AVG(total_usd) AS avg_revenue
FROM orders
GROUP BY month
ORDER BY month;
```

aov_change

```
SELECT
   year,
   month,
   SUM(total_usd) / COUNT(order_id) AS aov
FROM orders
GROUP BY year, month
ORDER BY year, month;
```

repeat_buyers

```
WITH repeat_customers AS (
   SELECT customer_id
   FROM orders
   GROUP BY customer_id
   HAVING COUNT(order_id) > 1
)
SELECT
   ROUND(
      COUNT(*) * 100.0 /
      (SELECT COUNT(DISTINCT customer_id) FROM customers),
```

```
        2
    ) AS repeat_buyer_percentage
FROM repeat_customers;
```

most_valuable_customers

```
SELECT
c.name,
c.customer_id,
c.country,
COUNT(o.order_id) AS total_orders,
ROUND(SUM(o.total_usd), 2) AS total_spent
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY
c.customer_id,name,
c.country
ORDER BY total_spent DESC
LIMIT 10;
```

product_revenue_by_category

```
SELECT
    p.category,
    ROUND(SUM(oi.quantity * oi.unit_price_usd), 2) AS revenue
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

best_selling

```
SELECT p.category, SUM(oi.quantity) AS sold
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY 1
ORDER BY 2 DESC;
```

highest_margin

```
SELECT
    p.name,
    ROUND(
        SUM((oi.unit_price_usd - p.cost_usd) * oi.quantity),
        2
    ) AS profit
FROM order_items oi
JOIN products p
```

```
    ON oi.product_id = p.product_id
GROUP BY p.name
ORDER BY profit DESC
LIMIT 10;
```

high_sales_low_margin

```
SELECT
    p.name,
    SUM(oi.quantity * oi.unit_price_usd) AS revenue,
    SUM((oi.unit_price_usd - p.cost_usd) * oi.quantity) AS profit,
    ROUND(
        SUM((oi.unit_price_usd - p.cost_usd) * oi.quantity) * 100.0 /
        SUM(oi.quantity * oi.unit_price_usd),
        2
    ) AS profit_margin_pct
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY p.name
ORDER BY revenue DESC
LIMIT 20;
```

revenue_by_source

```
SELECT
    source,
    ROUND(SUM(total_usd),2) AS revenue
FROM orders
GROUP BY source
ORDER BY revenue DESC;
```

customers_by_source

```
SELECT
    source,
    COUNT(DISTINCT customer_id) AS customers
FROM orders
GROUP BY source
ORDER BY customers DESC;
```

revenue_per_customer_by_source

```
SELECT
    source,
    ROUND(
        SUM(total_usd) /
        COUNT(DISTINCT customer_id),
        2
```

```
    ) AS revenue_per_customer
FROM orders
GROUP BY source
ORDER BY revenue_per_customer DESC;
```

aov_by_device

```
SELECT
    s.device,
    ROUND(AVG(o.total_usd),2) AS aov
FROM orders o
JOIN sessions s
    ON o.customer_id = s.customer_id
GROUP BY s.device
ORDER BY aov DESC;
```

rating_distribution

```
SELECT
    rating,
    COUNT(*) AS reviews
FROM reviews
GROUP BY rating
ORDER BY rating;
```

top_rated

```
SELECT
    p.category,
    ROUND(AVG(r.rating),2) AS avg_rating,
    COUNT(*) AS reviews
FROM reviews r
JOIN products p
    ON r.product_id = p.product_id
GROUP BY p.category
ORDER BY avg_rating DESC;
```

monthly_retention_rate

```
WITH first_purchase AS (
SELECT
customer_id,
MIN(order_time) AS first_order
FROM orders
GROUP BY customer_id
)
SELECT
YEAR(first_order) AS cohort_year,
```

```
COUNT(*) AS customers
FROM first_purchase
GROUP BY YEAR(first_order);
```

repeat_customers

```
SELECT
COUNT(DISTINCT customer_id) AS repeat_customers
FROM (
   SELECT customer_id
   FROM orders
   GROUP BY customer_id
   HAVING COUNT(order_id) > 1
) t;
```

retention_rate

```
SELECT
ROUND(
COUNT(DISTINCT CASE WHEN order_count > 1 THEN customer_id END)
*100.0/
COUNT(DISTINCT customer_id),
2
) AS retention_rate
FROM (
   SELECT
   customer_id,
   COUNT(order_id) AS order_count
   FROM orders
   GROUP BY customer_id
) t;
```

cohort_table

```
WITH cohorts AS (
SELECT
customer_id,
DATE_FORMAT(MIN(order_time),'%%Y-%%m') AS cohort_month
FROM orders
GROUP BY customer_id
)
SELECT *
FROM cohorts;
```

cohort_retention

```
WITH cohorts AS (
SELECT
```

```
customer_id,
DATE_FORMAT(MIN(order_time),'%%Y-%%m') AS cohort_month
FROM orders
GROUP BY customer_id
),
activity AS (
SELECT
o.customer_id,
c.cohort_month,
TIMESTAMPDIFF(
MONTH,
STR_TO_DATE(CONCAT(c.cohort_month,'-01'),'%%Y-%%m-%%d'),
o.order_time
) AS month_number
FROM orders o
JOIN cohorts c
ON o.customer_id=c.customer_id
)
SELECT
cohort_month,
month_number,
COUNT(DISTINCT customer_id) AS customers
FROM activity
GROUP BY cohort_month,month_number;
```

rfm

```
WITH last_year_orders AS (
   SELECT *
   FROM orders
   WHERE order_time >= DATE_SUB(
      (SELECT MAX(order_time) FROM orders),
      INTERVAL 1 YEAR
   )
),

rfm_base AS (
   SELECT
      customer_id,

      DATEDIFF(
         (SELECT MAX(order_time) FROM orders),
         MAX(order_time)
      ) AS recency,

      COUNT(order_id) AS frequency,

      ROUND(SUM(total_usd), 2) AS monetary
```

```
    FROM last_year_orders
    GROUP BY customer_id
),

rfm_scores AS (
    SELECT
        customer_id,
        recency,
        frequency,
        monetary,

        6 - NTILE(5) OVER (ORDER BY recency) AS r_score,
        NTILE(5) OVER (ORDER BY frequency) AS f_score,
        NTILE(5) OVER (ORDER BY monetary) AS m_score

    FROM rfm_base
)

SELECT
    CASE
    WHEN r_score >= 4
         AND f_score >= 4
         AND m_score >= 4
    THEN 'Best Customers'

    WHEN r_score >= 3
         AND f_score >= 3
    THEN 'Loyal Customers'

    WHEN r_score <= 2
         AND f_score >= 3
    THEN 'At Risk'

    WHEN r_score <= 2
         AND f_score <= 2
    THEN 'Lost Customers'

    ELSE 'Others'
END AS segment,
customer_id,
    recency,
    frequency,
    monetary,
    r_score,
    f_score,
    m_score,
    CONCAT(r_score, f_score, m_score) AS rfm_score
```

```
FROM rfm_scores
ORDER BY monetary DESC;
```